CECS 327 Sec 01 Distributed File System 1 Report

Group Members:

Dean Vu

Anthony Phan

Evan Gonzalez

Chord Integration

The system is built on top of a Chord style DHT that maps keys to nodes using consistent hashing. Each node is assigned a 160 bit identifier, and all keys are hashed into the same identifier space using SHA-1. The nodes are logically arranged in a ring, and each key is assigned to the node whose identifier is the successor of the key. All nodes are aware of the full ring membership, and routing is done by scanning the sorted node list to find the first node whose ID is greater than or equal to the key.

This approach ensures:

- uniform distribution of data across nodes
- deterministic placement of metadata and pages
- consistent routing for both reads and writes

Metadata and Page design

Our program represents each file using a metadata object, which contains a filename, the file size, number of pages and an ordered list of descriptors. Metadata is stored using a key derived from the filename, with each page being stored separately using a key based on the filename and the page number. The design of our program allows our system to reconstruct files using both the metadata and each page from the distributed nodes.

Distributed Sorting Strategy

The system performs distributed sorting by partitioning records across multiple nodes based on their keys. Our function preserves the order of operations and each record is routed to the node responsible for its mapped key and stores it in a local buffer. Before the sorting begins, all node buffers are going to be cleared to ensure no leftover data from previous operations. Each node is going to perform its own local sorting and then combine their results into a global sort. These final results are stored back into the Distributed File System.

Replication Strategy

Our system we have designed uses a replication strategy where each key is assigned to a group of nodes, each node having a leader and its immediate successors. The leader node is responsible for handling client requests and coordinating all updates. Non leader nodes forward requests given to them to their leader to handle if needed. For each operation given, the leader is going to send updates to all the replicas under its lead and require a majority to accept the operation before it is committed. This majority based replication improves our systems fault tolerance by ensuring our system will work fully before committing an update to its nodes.

Paxos message Flow

1. Leader receives write request
2. Leader generates:
    - ballot number
    - globally unique slot ID (string-based)
3. Leader sends:

    ACCEPT(slot, ballot, operation)

    to all replicas

4. Replicas:
    - check if ballot is valid
    - respond with ACCEPT if allowed
5. If majority accepts:
    - Leader sends:

        LEARN(slot, ballot, operation)
6. Replicas:
    - mark operation as learned
    - apply operation to storage

Failure Assumptions

The system assumes a crash-fault model:

- Nodes can stop responding (crash)
- Nodes do not behave maliciously (no Byzantine faults)
- Network communication is reliable but can experience delays

Fault tolerance:

- Each key is replicated on 3 nodes
- System tolerates failure of 1 node per group
- Majority quorum ensures continued operation

Limitations and Future Improvements

Limitations:

- No failure detection
- No log persistence across restarts
- No garbage collection for stale data
- Static membership

Future Improvements:

- Add dynamic node join/leave support
- Add persistent logging for recovery
- Introduce failure detection (heartbeats)
- Optimize sorting with parallel pipelines

Test Evidence

Sample input file:

```
--- Loading data.csv ---
0190,carol
0042,bob
0012,alice
0999,zack
0100,diana
0350,eve
```

Sorted output file:

```
--- Sorting into sorted_data.csv ---
0012,alice
0042,bob
0100,diana
0190,carol
0350,eve
0999,zack
```

Paxos messages:

```
--- Paxos / replication logs (tail) ---
Node 0 log tail:
  [19:26:29] node=0 ACCEPTED slot=1776738385274869300:3 ballot=7:3 op=PUT key=852705068422116942729130903394069444010218606569
  [19:26:33] node=0 LEARNED slot=1776738385274869300:3 ballot=7:3 op=PUT key=852705068422116942729130903394069444010218606569
  [19:26:33] node=0 APPLIED slot=1776738385274869300:3 op=PUT key=852705068422116942729130903394069444010218606569
  [19:26:37] node=0 LEADER begin slot=1776738397539773200:0 ballot=3:0 op=PUT key=12488166406708683759848937267188306746630875926l3 replicas=[0, 2923003274661805836407369665432566039311865
  [19:26:51] node=292300327466180583640736966543256603931186508595 LEARNED slot=1776738407780224100:1 ballot=8:1 op=PUT key=22400594631517113842716844840085615524388389853
  [19:26:51] node=292300327466180583640736966543256603931186508595 APPLIED slot=1776738407780224100:1 op=PUT key=22400594631517113842716844840085615524388389853
  [19:26:55] node=292300327466180583640736966543256603931186508595 COMMIT slot=1776738407780224100:1 ballot=8:1 key=22400594631517113842716844840085615524388389853
Node 584600654932361167281473933086513207862373017190 log tail:
  [19:26:19] node=584600654932361167281473933086513207862373017190 LEARNED slot=1776738373028633700:1 ballot=7:1 op=PUT key=22400594631517113842716844840085615524388389853
  [19:26:19] node=584600654932361167281473933086513207862373017190 APPLIED slot=1776738373028633700:1 op=PUT key=22400594631517113842716844840085615524388389853
  [19:26:41] node=584600654932361167281473933086513207862373017190 ACCEPTED slot=1776738397539773200:0 ballot=3:0 op=PUT key=12488166406708683759848937267188306746630875926l3
  [19:26:45] node=584600654932361167281473933086513207862373017190 LEARNED slot=1776738397539773200:0 ballot=3:0 op=PUT key=12488166406708683759848937267188306746630875926l3
  [19:26:45] node=584600654932361167281473933086513207862373017190 APPLIED slot=1776738397539773200:0 op=PUT key=12488166406708683759848937267188306746630875926l3
  [19:26:49] node=584600654932361167281473933086513207862373017190 ACCEPTED slot=1776738407780224100:1 ballot=8:1 op=PUT key=22400594631517113842716844840085615524388389853
  [19:26:53] node=584600654932361167281473933086513207862373017190 LEARNED slot=1776738407780224100:1 ballot=8:1 op=PUT key=22400594631517113842716844840085615524388389853
  [19:26:53] node=584600654932361167281473933086513207862373017190 APPLIED slot=1776738407780224100:1 op=PUT key=22400594631517113842716844840085615524388389853
```

Correctness check:

```
--- Validation ---
sorted_data.csv validated: True
```